



Software Relevant Tools, Standards, and Code Versioning using Github

Name:	John Cleif E. Paulino	Date:	22/08/2026
Program:	CPE	Section:	B3

A. Learning Outcomes:

At the end of this laboratory, students should be able to:

- 1. Identify software development tools and standards used in software projects.
- 2. Create and manage a GitHub repository.
- 3. Apply Git version control commands.
- 4. Collaborate using GitHub features.
- 5. Demonstrate proper documentation and coding standards.

B. Laboratory Scenario:

You have been hired as a junior software developer in a software company. Your team uses GitHub for source code management and follows coding standards to maintain software quality. Your task is to create a project repository, manage versions of source code, and document your work according to industry standards.

C. Practical Tasks:

1. Software Development Tools and Standards

Create a document named Tools_and_Standards.md containing:

- a. List at least 5 development tools.
- b. Describe the purpose of each tool.
- c. Identify at least three coding standards or best practices used in software development.

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

Software Development Tools

- 1. IntelliJ IDEA - IntelliJ IDEA is an integrated development environment mainly used for developing Java and other JVM-based applications. It provides code completion, debugging, project management, and testing features.
- 2. GitHub - GitHub is a platform for hosting and collaborating on software projects. Developers can store repositories, review code, manage issues, and collaborate through pull requests.
- 3. npm - npm is a package manager commonly used with JavaScript and Node.js. It allows developers to install, update, and manage external libraries and dependencies.
- 4. Chrome DevTools - Chrome DevTools is a collection of development tools built into Google Chrome. It helps developers inspect web pages, debug JavaScript, analyze network requests, and test website layouts.
- 5. Jenkins - Jenkins is an automation server used to automate tasks such as building, testing, and deploying software. It is commonly used as part of continuous integration and continuous delivery workflows.

Coding Standards and Best Practices

- 1. DRY (Don't Repeat Yourself) Avoid writing the same code multiple times. Reusable functions and components can reduce duplication and make software easier to maintain.
- 2. SOLID Principles SOLID principles provide guidelines for designing maintainable and flexible object-oriented software. They help developers organize code so that it is easier to modify and extend.
- 3. Code Documentation Important parts of a program should be documented clearly. Comments, documentation, and descriptive function names help other developers understand how the software works.

*Resize the image if necessary.

- 2. GitHub Repository Creation
 - a. Create a GitHub account (if none exists).
 - b. Create a public repository named: Software-Versioning-Lab
 - c. Add the following:
 - i. README.md
 - ii. LICENSE
 - iii. .gitignore
 - d. Upload this laboratory file in this repository once accomplished.
 - e. Submit GitHub repository link
 - f. Screenshot of repository homepage



Software Relevant Tools, Standards, and Code Versioning using Github

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

Software-Versioning-Lab

Public

Pin

Watch 0

Fork 0

Star 0

main

2 Branches

0 Tags

Go to file

Add file

Code

johncleifpaulino-web

Create Git_Report.md

8f89bf2 · 1 minute ago

13 Commits

.gitignore

Create .gitignore

4 hours ago

Git_Report.md

Create Git_Report.md

1 minute ago

LICENSE

Initial commit

4 hours ago

README.md

Update README.md

2 hours ago

README

MIT license

Software-Versioning-Lab

Paulino, John Cleif E. Software Design Laboratory / B3

About

No description, website, or topics provided.

Readme

MIT license

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

johncleifpaulino-web

*Resize the image if necessary.

3. Git Version Control

Perform the following Git commands and document them in a file named: (Git_Report.md)

Clone Repository

git clone <repository-url>

Creating New Branch

git checkout -b feature-update

Modify README.md

Add your name and laboratory information

Commit Changes

git add .

git commit -m "Updated README with laboratory information"

Push Changes

git push origin feature-update

Create Pull Request

feature-update → main

Screenshot the following:

Commands executed

Pull Request

Commit history

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

CPE106L-4 Software Design Laboratory

School of Artificial Intelligence, Electrical, Electronics, and Computer Engineering

Page | 2



Software Relevant Tools, Standards, and Code Versioning using Github

```

MINGW64/d/Software Design Laboratory/Software-Versioning-Lab
Admin@DESKTOP-FM76V9L MINGW64 ~
$ cd "D:\Software Design Laboratory"

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory
$ git clone "https://github.com/johncleifpaulino-web/Software-Versioning-Lab.git"
Cloning into 'Software-Versioning-Lab'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 22 (delta 3), reused 20 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (22/22), done.
Resolving deltas: 100% (3/3), done.

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory
$ cd Software-Versioning-Lab

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory/Software-Versioning-Lab (main)
$ git checkout -b feature-update
Switched to a new branch 'feature-update'

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory/Software-Versioning-Lab (feature-update)
$ git add .
git commit -m "Updated README with laboratory information"
Author identity unknown

*** Please tell me who you are.

Run

    git config --global user.email "you@example.com"
    git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Admin@DESKTOP-FM76V9L.(none)')

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory/Software-Versioning-Lab (feature-update)
$ git push origin feature-update
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (22/22), 9.92 KiB | 9.92 MiB/s, done.
Total 22 (delta 3), reused 22 (delta 3), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
remote:
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:   https://github.com/johncleifpaulino-web/Software-Versioning-Lab/pull/new/feature-update
remote:
To https://github.com/johncleifpaulino-web/Software-Versioning-Lab.git
 * [new branch]      feature-update -> feature-update

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory/Software-Versioning-Lab (feature-update)
$ ^C

Admin@DESKTOP-FM76V9L MINGW64 /d/Software Design Laboratory/Software-Versioning-Lab (feature-update)
$

```

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: **feature-update**

compare: **main**

 **Able to merge**. These branches can be automatically merged.

 **Add a title ***

Update README.md

Add a description

WritePreview

H B I          

Add your description here...

 **Markdown is supported**  **Paste, drop, or click to add files**

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

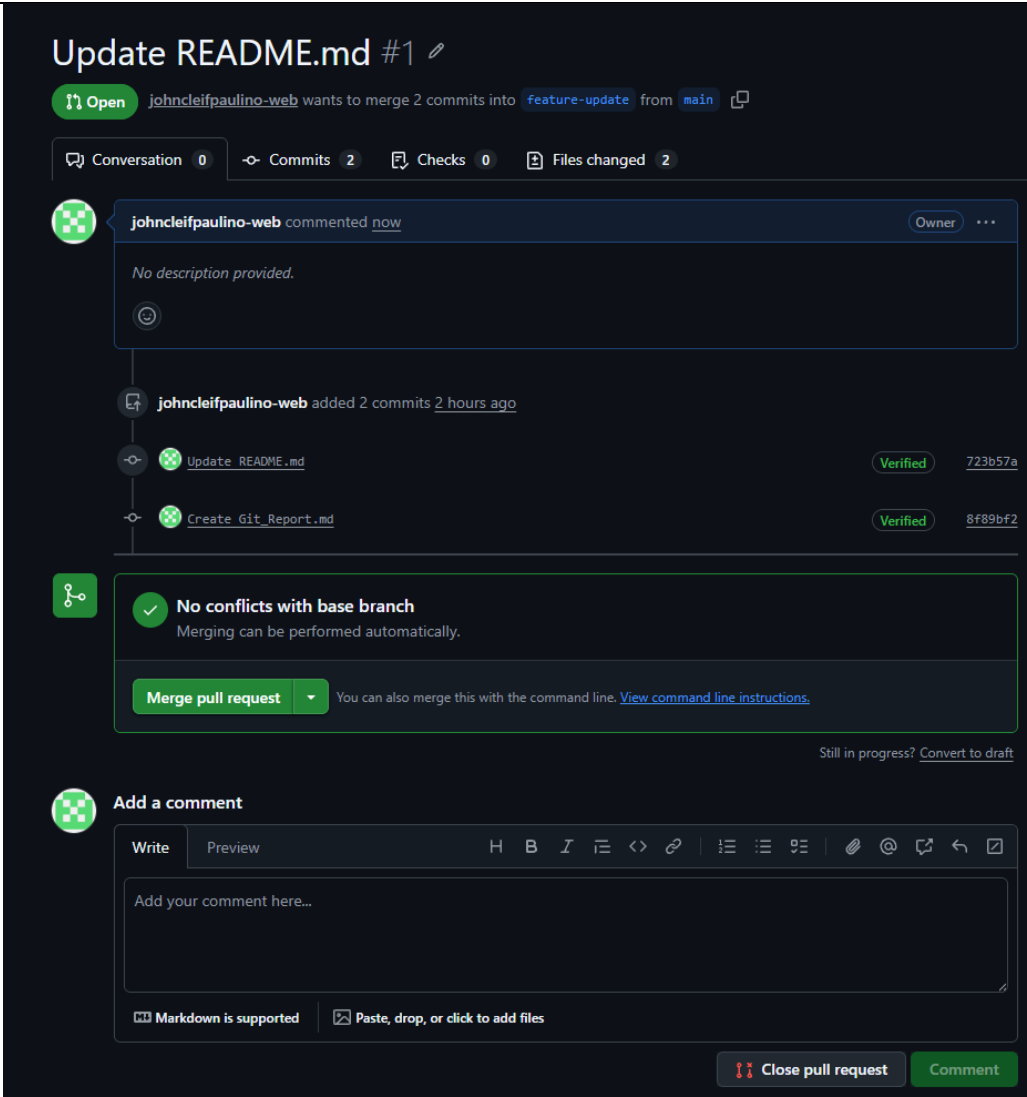
Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

Software Relevant Tools, Standards, and Code Versioning using Github



*Resize the image if necessary.

Software Relevant Tools, Standards, and Code Versioning using Github

D. Findings, Observations, and Comments:

Guide Questions:

- 1. What additional steps are taken in this activity?
- 2. Are there any difficulties during the GitHub setup?
- 3. What are your findings, observations, and comments in this exercise?

1. What additional steps are taken in this activity?

Additional steps include creating a separate branch, modifying the required files, staging and committing the changes, pushing the branch to GitHub, and creating a Pull Request. These steps allow the changes to be reviewed before being added to the main branch.

2. Are there any difficulties during the GitHub setup?

Yes. One difficulty encountered was configuring the Git username and email. The commit initially failed because Git could not identify the author. After configuring the Git identity, the changes could be committed successfully. Another challenge was understanding the difference between pushing a branch and creating a Pull Request.

3. What are your findings, observations, and comments in this exercise?

The exercise showed how Git and GitHub can be used to manage changes and collaborate on software projects. I observed that creating branches helps keep changes separate from the main branch, while commits provide a record of the changes made. I also learned that successfully pushing a branch does not automatically create a Pull Request or merge the changes into main. Overall, the activity

PICTURE/PROOF OF DOING THE ACTIVITY





Software Relevant Tools, Standards, and Code Versioning using Github

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	